```
T_k(n) = 2 * T_k(n/2) + O(n)
```

There are a few ways to solve this equation, for example the substitution method or telescoping, but the easiest is probably using master theorem[6]. All of these methods are out of the scope for this book, so we will just provide you with the solution, leaving to the curious reader working out the details:

```
T_k(n) = O(n * log(n))
```

In other words, the balanced construction method takes *linearitmic*[7] time.

To complete our analysis, if we look at the extra memory needed by this method, it will of course require `O(n)` memory for the tree. There is, however, more: if we don't partition the array in-place, and create a copy for each left and right sub-array, then each call to partition would use `O(n)` extra memory, and deriving a similar formula to the one for the running time, we could find out that also `M(n) = O(n * log(n))`.

Conversely, by partitioning the array in place, we can obtain the best possible result:

```
M_k(n) = O(n)
```

That's because we would need only a constant amount of memory for the internals of the function, plus the `O(n)`